

Question: Design a Pluggable Rate Limiting System for External Resource Usage

You are building a backend system that serves client API requests.

Flow:

1. A client calls one of your APIs.
2. The API forwards the request to an internal service.
3. The internal service may or may not call an external resource depending on business logic.
4. The external resource is paid, and you are charged based on usage.

Your task is to design and implement a **rate limiting system for the external resource call**, not for the incoming client API itself.

That means:

- Not every API request consumes quota.
- Rate limiting should be applied only at the point where the system is about to call the external resource.

Requirements

Design a rate limiting module with the following requirements:

Functional Requirements

1. The system should decide whether a particular external call is allowed or denied.
2. The rate limiter should support multiple rate limiting algorithms.
3. At minimum, implement:
 - **Fixed Window Counter**
 - **Sliding Window Counter**
4. The design should allow plugging in any future algorithm easily, such as:
 - Token Bucket
 - Leaky Bucket
 - Sliding Log
5. The system should support configuring limits such as:
 - 100 requests per minute

- 1000 requests per hour
- 6. The rate limiting key can vary based on use case, for example:
 - per customer
 - per tenant
 - per API key
 - per external provider
- 7. The module should expose a simple interface that internal services can use before making the external call.

Non-Functional Requirements

1. The design should be extensible.
2. It should follow good OOP and SOLID principles.
3. It should be thread-safe.
4. The implementation should be efficient in terms of time and space.
5. The code should be easy to test.

Deliverables

1. Design the classes and interfaces required.
2. Write code for the system.
3. Implement:
 - Fixed Window Counter
 - Sliding Window Counter
4. Show how a caller can switch between algorithms without changing business logic.
5. Explain key design decisions.
6. Discuss trade-offs between the two algorithms.

Example Use Case

Suppose user **T1** is allowed to make at most **5 external calls per minute**.

For each client request:

- business logic runs first
- if no external call is needed, no rate limiting check is performed
- if an external call is needed, the rate limiter is consulted
- if allowed, the external API is called
- otherwise, the request is rejected or handled gracefully